



▢ Módulo 4 · Introducción a APIs de IA

APIs de IA — Seguridad, Diseño de Apps y Pipeline Integrado

Ejercicio de Clase · Ejercicio 3 de 3 · Bloques 4.4 — 4.5 — 4.8 — Integración Pipeline

Nombre del alumno/a	Angel Echenique
Fecha	11/04/2026
▢ Enlace GitHub del repositorio	

▢ **Instrucciones:** Ejercicio práctico. Diseña soluciones reales usando APIs de IA integrando lo que ya sabes de modularización y GitHub Actions. Razona tus decisiones técnicas en cada bloque.

BLOQUE 1 — Seguridad: buenas prácticas y errores comunes (4.8)

En clase vimos los errores más frecuentes y las buenas prácticas al trabajar con APIs de IA. Completa la tabla:

Buenas Prácticas ✓	Errores Comunes ✗
Nunca hardcodear la API key en el código fuente	Subir la API key directamente en el código a GitHub
Nunca hardcodear API keys Las claves no deben aparecer en el código fuente ni en repositorios. Evita exponerlas en GitHub, capturas o logs.	**No proteger las API keys** Suben claves a GitHub, las dejan en el código o las comparten sin querer. Es el fallo más frecuente.
Usar variables de entorno Las keys deben cargarse desde el sistema operativo o un gestor seguro (dotenv, secrets manager). Mantiene el código limpio y seguro.	**No controlar costos** No ponen límites, no monitorizan y usan modelos caros sin necesidad. Resultado: facturas inesperadas.
Añadir `.env` al `.gitignore` Evita que los archivos con claves o configuraciones sensibles se suban al repositorio.	**Usar modelos demasiado grandes** Eligen modelos "top" para tareas simples. Más tokens, más coste y más latencia sin aportar valor.
Rotar keys periódicamente Cambiar las claves reduce el impacto si alguna se filtra. Es una práctica estándar en seguridad.	**No entender tokens ni límites** No calculan el coste por token, no conocen el context window y saturan el modelo con prompts enormes.

 Limitar uso por entorno(dev/staging/prod) Cada entorno debe tener su propia key con permisos y límites distintos. Minimiza daños en caso de abuso.	 **No manejar errores de red** Asumen que la API siempre responde. Falta de `try/except`, reintentos o gestión de timeouts.
 Monitorizar consumo y costos Revisar dashboards y alertas evita sorpresas en facturación y detecta usos anómalos.	 **No validar respuestas de la IA** Confían ciegamente en la salida del modelo. No comprueban formato, contenido ni coherencia antes de usarlo.
 Implementar rate limiting Protege tu API de abusos, bucles infinitos o ataques. Evita que un error dispare miles de llamadas.	

1.2 — Detecta el error de seguridad

Lee el siguiente fragmento de código. Identifica TODOS los problemas de seguridad que encuentras y cómo los corregirías:

```
# archivo: app.py - subido a GitHub público
from openai import OpenAI

API_KEY = 'sk-proj-abc123xyz789...'
client = OpenAI(api_key=API_KEY)

def responder(pregunta_usuario):
    while True: # bucle sin límite
        response = client.chat.completions.create(
            model='gpt-4o', # modelo más caro
            messages=[{'role': 'user', 'content': pregunta_usuario}]
        )
        print(response.choices[0].message.content)
```

Problema detectado	¿Cómo lo corregirías?
API key hardcodeda en el código	Usar <code>os.getenv('OPENAI_API_KEY')</code>
Bucle infinito <code>while True</code> sin límite	Eliminar el bucle o agregar condición de salida
Modelo muy caro (gpt-4o)	Usar <code>gpt-4o-mini</code> para tareas simples
Sin manejo de errores	Agregar <code>try/except</code> con reintentos
Subido a GitHub público	Meter la key en GitHub Secrets
Sin rate limiting	Agregar límite de llamadas
Sin validación de respuesta	Verificar que <code>response.choices[0].message.content</code> no sea None

1.3 — Variables de entorno y GitHub Secrets

Ordena del 1 al 6 los pasos correctos para gestionar una API key de forma segura en un proyecto con GitHub Actions:

Orden	Paso a seguir
2	Añadir el archivo .env al .gitignore para que nunca se suba al repositorio.
1	Crear el archivo .env local con OPENAI_API_KEY=sk-proj-tu-clave.
3	En el código, leer la clave con os.getenv('OPENAI_API_KEY').
4	En GitHub, ir a Settings → Secrets and variables → Actions y añadir el Secret.
5	En el archivo .yaml del workflow, pasar el Secret como variable de entorno al step.
6	Verificar en el pipeline que la variable está disponible antes de ejecutar el script.

BLOQUE 2 — Diseña una aplicación real con APIs de IA (4.4 + 4.5)

□ Caso práctico: Tu equipo quiere construir un sistema de soporte técnico automatizado para una empresa de software. El sistema debe: (1) clasificar tickets de soporte por categoría y urgencia, (2) generar una respuesta inicial automática, (3) detectar si el problema es conocido buscando en la base de conocimiento por significado. Tienes un presupuesto limitado y el sistema recibirá ~500 tickets/día.

Diseña la arquitectura de la aplicación completando la siguiente tabla:

Funcionalidad	API / Proveedor elegido + Modelo	Justificación técnica
Clasificación de tickets (categoría + urgencia)	OpenAI GPT-4o-mini	Para clasificación de texto, no se necesita el modelo más potente. GPT-4o-mini tiene un coste de 0.002/1Ktokensinputy0.002/1K tokensinputy 0.004/1K output. Con 200 tokens de entrada (descripción del ticket) + ~50 de salida (categoría + prioridad), cada clasificación cuesta aproximadamente 0.0005. Para 500 tickets/día=**0.0005. Para 500 tickets/día=**0.25/día** (\$7.50/mes). Es suficiente porque la tarea es estructurada y objetiva, no requiere creatividad.

Generación de respuesta inicial automática	Google Gemini 1.5 Flash	El modelo gratuito de Google tiene un coste muy bajo o nulo para volúmenes moderados. Generar una respuesta inicial de ~100-150 tokens (2-3 frases) con un tone profesional y empático. La ventaja es que Flash está optimizado para velocidad, ideal para dar respuestas en tiempo real al usuario. Alternativa: GPT-4o-mini si se quiere consistencia en el estilo, coste ~\$0.001/ticket.
Búsqueda semántica en base de conocimiento	OpenAI text-embedding-3-small	Este modelo genera embeddings de 1536 dimensiones a 0.00002/1Ktokens(20xmás barato que embedding-ada-002). Para buscar en la base de conocimiento, convertimos cada documento a embedding una sola vez y lo guardamos. Cada búsqueda = embedding del query (50 tokens) + similitud coseno. Coste por búsqueda: 0.00002/1Ktokens(20x más barato que embedding-ada-002). Para buscar en la base de conocimiento, convertimos cada documento a embedding una sola vez y lo guardamos. Cada búsqueda = embedding del query (50 tokens) + similitud coseno. Coste por búsqueda: 0.000001. Si la base tiene 1000 artículos, el coste de indexación es ~\$0.02 una sola vez.
Análisis de capturas de pantalla adjuntas	Google Gemini 1.5 Pro	Gemini 1.5 Pro acepta hasta 2 millones de tokens de contexto y procesa imágenes nativamente. Permite analizar capturas de errores, mensajes de aplicación, logs, etc. GPT-4o también lo hace pero a mayor coste. Eligimos Gemini porque tiene la mejor relación calidad-precio en multimodalidad y puede procesar imágenes grandes sin reducir calidad. Coste: ~\$0.00125/1K tokens input (incluye imagen).

Coste total diario estiando

Función	Tokens/día	Coste unitario	Coste diario
Clasificación	$500 \times 250 = 125K$	\$0.002/1K	\$0.25
Respuestas	$500 \times 150 = 75K$	\$0.0001/1K (Flash)	\$0.0075
Búsqueda semántica	$500 \times 50 = 25K$	\$0.00002/1K	\$0.0005
Análisis de imágenes	$\sim 100 \text{ tickets} \times 2K$	\$0.00125/1K	\$0.25
TOTAL		-0.51/día = 15/mes	

2.2 — Diseño modular de la aplicación

Usando lo que ya sabes de modularización, divide esta aplicación en módulos. Para cada módulo indica su responsabilidad y qué función de la API utiliza:

Nombre del módulo	Responsabilidad única (SRP)	Llamada a la API que realiza
<code>clasificador.py</code>	Analiza el texto del ticket y determina categoría (técnico/facturación/cuenta/otro) y prioridad (alta/media/baja). Solo hace clasificación.	<code>client.chat.completions.create()</code> con system prompt especializado en classification y few-shot examples
<code>generador_respuestas.py</code>	Genera una respuesta inicial personalizada basada en la categoría y prioridad del ticket. No clasifica, solo genera contenido.	<code>client.chat.completions.create()</code> con template dinámico según categoría
<code>buscador.py</code>	Recibe la consulta del problema y busca en la base de conocimiento usando embeddings. Devuelve documentos similares.	<code>client.embeddings.create()</code> para generar embedding de query + cálculo de similitud coseno

<code>analizador_imagenes.py</code>	Recibe una imagen (captura de pantalla) y la analiza para detectar errores, texto relevante o información adicional.	<code>client.chat.completions.create()</code> con imagen en formato base64 (multimodal)
<code>orquestador.py</code>	Coordina los módulos anteriores. Recibe el ticket completo, llama a clasificador → buscador → generador → (opcional) analizador. Devuelve el resultado final.	No hace llamadas directas, orchestration de los otros módulos

Archivo 2.2 agregado con la demas informacion de este punto.

2.3 — Optimización de costes

Con 500 tickets/día, el coste puede dispararse. Describe 4 estrategias concretas para optimizar el coste de esta aplicación:

Nº	Estrategia de optimización	¿Cómo se implementa en el código/pipeline?
1	Usar el modelo adecuado para cada tarea	GPT-4o-mini para clasificación (\$0.002/1K), Gemini Flash para respuestas cortas, embeddings-3-small para búsqueda. No usar GPT-4o completo salvo para análisis complejos de imágenes.
2	Limitar max_tokens en cada módulo	clasificador: max_tokens=100 (solo necesita categoría + prioridad). generador_respuestas: max_tokens=200 (respuesta corta inicial). buscador: sin salida de IA, solo cálculo local. analizador_imagenes: max_tokens=300.
3	Cachear respuestas y base de conocimiento	Los embeddings de la base de conocimiento se calculan una sola vez al iniciar (no en cada ticket). Respuestas frecuentes se almacenan en caché. Usar Redis o diccionario en memoria.
4	Rate limiting y límites por pipeline	En el workflow de GitHub Actions, limitar a máximo 100 llamadas/minuto. Implementar en código: <code>time.sleep(0.5)</code> entre llamadas. Establecer budget alerts en OpenAI (\$10/mes máximo).
5	Comprimir prompts	Eliminar espacios innecesarios, usar nombres cortos en few-shot, no 重复 información. Un prompt de 500 tokens puede reducirse a 300 sin perder funcionalidad.

Concepto	Sin optimizar	Con optimización	Ahorro
Clasificación (500/día)	\$0.25/día	\$0.15/día (tokens reducidos)	40%
Respuestas (500/día)	\$0.20/día	\$0.05/día (Flash + cache)	75%
Búsqueda semántica	\$0.10/día	\$0.01/día (embeddings cacheados)	90%
Análisis imágenes (100/día)	\$0.50/día	\$0.25/día (max_tokens=300)	50%
TOTAL	1.05/día = 32/mes	0.46/día = 14/mes	56%

Codigo optimizado en el archivo que 2.3 que adjunte.

BLOQUE 3 — Pipeline CI/CD integrado con APIs de IA (GitHub Actions)

Ya sabes diseñar workflows con GitHub Actions. Ahora integra una API de IA en el pipeline de la aplicación de soporte del Bloque 2.

3.1 — Diseña el workflow de GitHub Actions

Describe los steps del workflow .yml que automatizaría el despliegue y la integración de la API de IA. NO necesitas escribir el YAML exacto, solo describe cada step y su propósito:

Step	Nombre del paso	¿Qué hace?	¿Por qué es necesario?
1	Checkout	Descarga el código del repositorio con actions/checkout@v4	Permite acceder a los archivos del proyecto para ejecutarlos
2	Setup Python	Instala Python 3.11 con actions/setup-python@v5	El código está en Python, necesita el intérprete para ejecutarse
3	Cache dependencies	Guarda en caché ~/.cache/pip para acelerar instalaciones	Evita reinstalar numpy, openai, etc. en cada ejecución
4	Install dependencies	Ejecuta pip install -r requirements.txt	Instala las librerías necesarias: openai, sklearn, python-dotenv
5	Configurar Secrets	Pasa OPENAI_API_KEY: \${ secrets.OPENAI_API_KEY } como variable de entorno	El código necesita la API key para llamar a OpenAI/Gemini
6	Run tests	Ejecuta pytest tests/ con continue-on-error: false	Verifica que el código pasa los tests antes de continuar
7	Ejecutar aplicación	Corre python main.py o el orquestador	Procesa tickets de prueba y verifica integración con IA
8	Reporte de Ergebnisse	Genera un resumen del pipeline (opcional: Slack/Discord)	Feedback visual del estado del pipeline

Ejemplo adjuntado en el archivo 3.1 pipeline

3.2 — Verdadero o Falso: Pipeline con APIs de IA

Marca V (Verdadero) o F (Falso) y justifica brevemente las que sean Falsas:

V / F	Afirmación	Justificación si es Falsa
v	Las API keys deben almacenarse como GitHub Secrets, nunca en el código ni en el .yaml directamente.	
v	Un pipeline puede disparar miles de llamadas a la API si hay un bucle infinito o una mala configuración.	
v	El verdadero reto del coste no es el precio por llamada, sino controlar el uso descontrolado.	
v	Si el proveedor de IA sufre una caída, el pipeline con manejo de errores puede hacer reintento automático.	
v	Gemini Flash y GPT-4o-mini son los modelos más recomendados para pipelines de alta frecuencia.	
f	Una vez integrada la API en el pipeline, ya no hace falta monitorizar el consumo.	Incorrecto. El consumo debe monitorizarse siempre mediante dashboards y alertas para detectar usages anómalos, prevenir facturas inesperadas y optimizar costes.

Explicaciones detalladas de las verdaderas

1. API keys en GitHub Secrets

Las API keys nunca deben estar en el código ni en logs. GitHub Secrets cifra las claves y las inyecta como variables de entorno solo en runtime.

2. Riesgo de bucle infinito

Un while True o un test mal configurado puede generar miles de llamadas. Ejemplo: 500 tickets × 10 reintentos = 5000 llamadas en minutos. Coste: ~\$10 en lugar de \$0.50.

3. Coste controlado

El precio por llamada es bajo (\$0.0004), pero con volumen alto puede subir rápido. 10,000 llamadas/día = ~\$4/día = \$120/mes.

4. Manejo de errores

Con try/except y retry podemos manejar 5xx de OpenAI. Pero rate limits (429) deben manejarse con backoff exponencial.

5. Modelos recomendados

GPT-4o-mini y Gemini Flash son los más económicos. Para pipelines de alta frecuencia, no tiene sentido usar GPT-4o a \$0.01/call cuando mini cuesta 20x menos.

BLOQUE 4 — Manejo de errores en llamadas a la API (4.8)

Las APIs de IA pueden fallar por múltiples razones. Completa la tabla indicando qué causa cada error y cómo lo manejarías en el código:

Tipo de error	¿Qué lo causa?	¿Cómo lo manejarías?
Rate limit (429)	Has hecho demasiadas peticiones en poco tiempo. El proveedor te rechaza.	Implementar backoff exponencial : esperar 1s, 2s, 4s, 8s... entre reintentos. Also configurar rate limiting en el código.
Timeout del servidor	El proveedor tarda más de lo esperado en responder (模型 sobrecargado).	Agregar timeout en la llamada (ej: 30s). Reintentar hasta 3 veces con try/except y time.sleep() entre intentos.
Error de autenticación (401)	API key inválida, expirada o mal configurada.	Verificar que la key existe en os.getenv(). Revocar la key old y crear una nueva en la consola del proveedor.
Prompt demasiado largo (400)	El texto supera el context window del modelo.	Reducir el prompt: truncar historial, usar fewer tokens, dividir en partes. Verificar longitud antes de enviar.
Error del servidor del proveedor (5xx)	Fallo interno en OpenAI/Gemini/Claude. No es tu culpa.	Reintento automático con backoff. Notificar al equipo. Revisar status page del proveedor.
Respuesta en formato inesperado	La IA no devolvió lo que esperábamos (ej: no sea JSON válido).	Validar con try/except json.JSONDecodeError. Definir respuesta por defecto si falla.

Adjunto archivo del bloque 4

BLOQUE 5 — Proyecto integrador final

□ Situación real: Eres el desarrollador principal de una startup que quiere lanzar en 2 semanas un MVP (producto mínimo viable) de un asistente de IA para gestión de contratos legales. Debe: analizar contratos PDF, detectar cláusulas de riesgo, responder preguntas sobre el contrato, y enviar un resumen ejecutivo por email. La empresa ya usa GitHub y tiene conocimientos de Python y GitHub Actions.

Diseña tu estrategia técnica completa respondiendo estas preguntas clave:

Decisión técnica	Tu respuesta justificada
¿Qué proveedor y modelo elegirías para analizar contratos largos? ¿Por qué?	Adjunto archivo del bloque 5.
¿Qué parámetros clave configurarías (temperature, max_tokens, system prompt)?	Adjunto archivo del bloque 5.
¿Cómo estructurarías los módulos de la aplicación?	Adjunto archivo del bloque 5.
¿Cómo gestionarías la API key en el repositorio y en el pipeline?	Adjunto archivo del bloque 5.
¿Qué harías para controlar que el coste no se dispare?	Adjunto archivo del bloque 5.
¿Qué errores anticipas y cómo los manejarías?	Adjunto archivo del bloque 5.